

Formation Engine Methodology

Section 2 — Information and State Ontology

Formation Engine Development Team

Version 3.0.0 — First Principles Draft

1 The State as the Fundamental Object

In computational physics, the choice of how to represent system state determines nearly every downstream design decision. The formation engine is built on a single principle: there exists one canonical data structure — the **State** — that completely describes the physical system at any instant.

This is not a software engineering decision. It is a reflection of the physics: classical mechanics operates on phase space, the space of all possible positions and momenta. The State is the discrete representation of a point in this continuous space.

Every operation in the framework — force evaluation, time integration, energy computation, structural analysis, file I/O — reads from or writes to this structure. Nothing physical exists outside of it. No hidden global variables. No implicit caches. No side-channel state.

This constraint has profound consequences:

- **Reproducibility:** Given a State at time t , the State at time $t + \Delta t$ is uniquely determined by the equations of motion.
- **Testability:** Any physical operation can be tested by comparing States before and after.
- **Serializability:** The State can be written to disk and restored exactly, enabling checkpointing and provenance tracking.
- **Composability:** Operations can be chained: each takes a State and returns a State.

This section defines the State formally, specifies what each field means physically, and explains how States are persisted through the file format hierarchy.

1.1 Formal Definition

A State $\mathcal{S}$ at time t is the tuple:

$$\mathcal{S}(t) = (N, \mathbf{M}, \boldsymbol{\tau}, \mathbf{X}(t), \mathbf{V}(t), \mathbf{Q}(t), \mathbf{B}(t), \mathbf{F}(t), \mathcal{E}(t), \mathcal{L}, \mathcal{B}) \quad (1)$$

where each component is defined as:

Symbol	Type	Description
N	$\mathbb{Z}^+$	Number of particles
$\mathbf{M}$	$\mathbb{R}^N$	Masses (amu)
$\boldsymbol{\tau}$	$\mathbb{Z}^N$	Atomic numbers (element identities)
$\mathbf{X}(t)$	$\mathbb{R}^{N \times 3}$	Positions (Å)
$\mathbf{V}(t)$	$\mathbb{R}^{N \times 3}$	Velocities (Å/fs)
$\mathbf{Q}(t)$	$\mathbb{R}^N$	Partial charges (e)
$\mathbf{B}(t)$	Edge set	Bond graph (pairs of atom indices)
$\mathbf{F}(t)$	$\mathbb{R}^{N \times 3}$	Forces (kcal/(mol·Å))
$\mathcal{E}(t)$	$\mathbb{R}^6$	Energy ledger (kcal/mol)
$\mathcal{L}$	List	Event log (metadata)
$\mathcal{B}$	$\mathbb{R}^3 \times \{0, 1\}$	Periodic box + enable flag

The explicit time dependence $\mathcal{S}(t)$ indicates that some fields evolve during dynamics. However, not all components change at the same rate or for the same reasons. This motivates the tripartite classification.

1.2 Tripartite Classification: Identity, Phase, Scratch

The State components partition into three categories based on their mutability and physical meaning.

1.2.1 Identity (Immutable)

The identity components define *what* the system is:

$$N : \text{Number of particles (fixed at construction)} \quad (2)$$

$$\mathbf{M} : \text{Atomic masses (set from periodic table, never modified)} \quad (3)$$

$$\boldsymbol{\tau} : \text{Element identities (defines chemical composition)} \quad (4)$$

These fields are set once during initialization and never change. No integrator, no force evaluator, no analysis tool may modify them. This is not a performance optimization — it is a physical constraint. The number of particles in a classical system does not change spontaneously. The mass of an atom is an intrinsic property. The element identity is defined by the nuclear charge.

In practice, this means:

- N is a compile-time constant for static-size systems or a runtime constant for dynamic-size systems
- $\mathbf{M}$ is populated from the periodic table via a lookup: $M_i = \text{AtomicWeight}(\tau_i)$
- $\boldsymbol{\tau}$ is read from the input file (e.g., `H`, `C`, `O`) and converted to atomic numbers

Attempting to modify these fields after initialization is a compile error in C++ (via `const` correctness) or a runtime error in Python (via property setters that raise exceptions).

1.2.2 Phase (Dynamic)

The phase components define *where* the system is in configuration space:

$$\mathbf{X}(t) : \text{Positions (evolve under equations of motion)} \quad (5)$$

$$\mathbf{V}(t) : \text{Velocities (evolve under equations of motion)} \quad (6)$$

$$\mathbf{Q}(t) : \text{Partial charges (formally dynamic, typically static)} \quad (7)$$

$$\mathbf{B}(t) : \text{Bond graph (currently static, future: reactive)} \quad (8)$$

These are the degrees of freedom. In Hamiltonian mechanics, the phase space is the space of all possible $(\mathbf{X}, \mathbf{V})$ pairs. The charges $\mathbf{Q}$ are formally part of the phase space to permit future charge-equilibration models (e.g., QEq, fluctuating charge methods) but are currently treated as parameters rather than dynamical variables.

The bond graph $\mathbf{B}$ is an edge set:

$$\mathbf{B} = \{(i, j) \mid i < j, \text{ atoms } i \text{ and } j \text{ are bonded}\} \quad (9)$$

Bonds are currently inferred from geometry via distance criteria and held fixed during dynamics. Section 8 describes how the reaction engine proposes bond changes *between* simulation runs.

1.2.3 Scratch (Recomputed)

The scratch components are derived quantities that can be regenerated from the phase:

$$\mathbf{F}(t) : \text{Forces (computed from } \mathbf{X}(t) \text{ via force field)} \quad (10)$$

$$\mathcal{E}(t) : \text{Energy ledger (computed from } \mathbf{X}(t), \mathbf{Q}(t), \mathbf{B}(t)) \quad (11)$$

$$\mathcal{L} : \text{Event log (append-only metadata)} \quad (12)$$

$$\mathcal{B} : \text{Periodic box (boundary conditions)} \quad (13)$$

These are *not* persistent state in the thermodynamic sense. They are computational artifacts. Forces are the negative gradient of the potential:

$$\mathbf{F}_i = -\nabla_{\mathbf{x}_i} U(\mathbf{X}, \mathbf{Q}, \mathbf{B}) \quad (14)$$

This is a pure function of the positions (and charges and topology). No history dependence. No hysteresis. Given $\mathbf{X}$, $\mathbf{F}$ is uniquely determined.

The distinction matters for two reasons:

Checkpointing To save the State to disk, only the identity and phase components need to be written. The scratch components can be regenerated on restore. This reduces file sizes and prevents inconsistency (e.g., storing forces that don't match the positions).

Validation To verify that a force model is implemented correctly, compute $\mathbf{F}$ from $\mathbf{X}$, perturb $\mathbf{X}$ slightly, recompute $\mathbf{F}$, and check that the finite-difference gradient matches the analytic force. This test is only valid if $\mathbf{F}$ is *purely* a function of $\mathbf{X}$.

The event log $\mathcal{L}$ is a special case: it is metadata (convergence flags, warnings, provenance notes) that does not affect physics but is useful for post-processing. It is append-only and never read by physics code.

The periodic box $\mathcal{B}$ specifies boundary conditions:

$$\mathcal{B} = (L_x, L_y, L_z, \text{enabled}) \quad (15)$$

For non-periodic systems, `enabled` = 0. For periodic systems, the box dimensions define the fundamental domain $[0, L_x) \times [0, L_y) \times [0, L_z)$.

1.3 Why This Decomposition Is Not Obvious

Many simulation codes conflate these categories. Common anti-patterns:

Forces stored as State Some frameworks store forces as persistent fields, updated incrementally during integration. This creates opportunities for inconsistency: if the force evaluation is modified (e.g., changing the cutoff radius), old force values may linger. The formation engine zeroes $\mathbf{F}$ before every evaluation, ensuring it always matches the current positions.

Topology as input Some frameworks read bond tables from files and assume they are correct. The formation engine *infers* $\mathbf{B}$ from $\mathbf{X}$ using distance criteria. This makes the bond graph a derived quantity, not an input. Section 2.5 explains the inference rules.

Masses as parameters Some frameworks allow users to specify masses independently of element types. This permits unphysical systems (e.g., a carbon atom with the mass of uranium). The formation engine locks $\mathbf{M}$ to $\boldsymbol{\tau}$ via the periodic table, making it impossible to create mass-inconsistent states.

These constraints may seem restrictive, but they prevent entire classes of errors. The State is designed to be *self-consistent by construction*.

2 The Energy Ledger

The potential energy U is not a single scalar. It is a sum of physically distinct contributions:

$$U = U_{\text{bond}} + U_{\text{angle}} + U_{\text{torsion}} + U_{\text{vdW}} + U_{\text{Coulomb}} + U_{\text{external}} \quad (16)$$

The framework requires that every force model report its contribution to each term. The energy ledger $\mathcal{E}$ is a 6-component vector:

$$\mathcal{E} = \begin{pmatrix} U_{\text{bond}} \\ U_{\text{angle}} \\ U_{\text{torsion}} \\ U_{\text{vdW}} \\ U_{\text{Coulomb}} \\ U_{\text{external}} \end{pmatrix} \quad (17)$$

This is not bookkeeping. It is physics. The decomposition reveals *why* a structure formed the way it did.

2.1 Physical Interpretation

Consider two formations of the same composition:

Formation A:

$$\begin{aligned}U_{\text{bond}} &= -50.0 \text{ kcal/mol} \\U_{\text{vdW}} &= -2.0 \text{ kcal/mol} \\U_{\text{Coulomb}} &= 0.0 \text{ kcal/mol} \\U_{\text{total}} &= -52.0 \text{ kcal/mol}\end{aligned}$$

Formation B:

$$\begin{aligned}U_{\text{bond}} &= -10.0 \text{ kcal/mol} \\U_{\text{vdW}} &= -35.0 \text{ kcal/mol} \\U_{\text{Coulomb}} &= -7.0 \text{ kcal/mol} \\U_{\text{total}} &= -52.0 \text{ kcal/mol}\end{aligned}$$

Both have the same total energy. But their physical character is entirely different:

- Formation A is a **covalently bonded molecule**. The bonding energy dominates. Disrupting this structure requires breaking chemical bonds.
- Formation B is a **molecular cluster**. Van der Waals and electrostatic interactions dominate. This structure can dissociate by thermal fluctuations at modest temperature.

Without the energy decomposition, these two formations would be indistinguishable. The ledger makes the physics transparent.

2.2 Implementation Requirement

Every force model implements the interface:

$$\text{eval} : \mathcal{S} \rightarrow (\mathbf{F}, \mathcal{E}) \tag{18}$$

The model must:

1. Zero the ledger: $\mathcal{E} \leftarrow \mathbf{0}$
2. Compute forces and accumulate energies: $\mathbf{F} \leftarrow \mathbf{F} + \mathbf{f}_{\text{contribution}}, \mathcal{E}_k \leftarrow \mathcal{E}_k + U_{\text{contribution}}$
3. Return $(\mathbf{F}, \mathcal{E})$

The zeroing step is mandatory. A model that fails to zero the ledger before accumulation produces drift: energies from previous evaluations persist and contaminate the current result.

For composite models (e.g., bonded + nonbonded), each sub-model contributes to the appropriate ledger entry:

- Harmonic bonds $\rightarrow U_{\text{bond}}$
- Harmonic angles $\rightarrow U_{\text{angle}}$
- Periodic torsions $\rightarrow U_{\text{torsion}}$
- Lennard-Jones $\rightarrow U_{\text{vdW}}$
- Coulomb $\rightarrow U_{\text{Coulomb}}$

External fields (restraints, walls, applied forces) contribute to U_{external} .

2.3 Kinetic Energy: Not Part of the Ledger

Kinetic energy K is computed from velocities:

$$K = \sum_{i=1}^N \frac{1}{2} m_i |\mathbf{v}_i|^2 \quad (19)$$

It is *not* part of $\mathcal{E}$. The ledger contains only potential energy contributions. Total energy $E = K + U$ is computed on demand:

$$E = K(\mathbf{V}) + \sum_{k=1}^6 \mathcal{E}_k \quad (20)$$

This distinction matters for thermostatted dynamics. In NVT (canonical) ensemble, the system exchanges energy with a heat bath. Total energy E fluctuates by design. But the potential energy decomposition remains meaningful: it describes the instantaneous force balance, independent of kinetic effects.

3 State Persistence: The File Hierarchy

States must be serializable. The framework defines a layered file format hierarchy where each level adds information without modifying the previous level.

3.1 Design Principle: Strict Additivity

The hierarchy is:

$$\text{xyzZ} \subset \text{xyzA} \subset \text{xyzC} \subset \text{xyzF} \quad (21)$$

where $\subset$ denotes “is a subset of”: every xyzZ file is a valid xyzA file, every xyzA is a valid xyzC, and so on. This means:

- A parser for xyzF can read xyzC, xyzA, or xyzZ
- Adding metadata (e.g., bond table) never breaks backward compatibility
- Legacy tools that expect bare XYZ can still read xyzA/xyzC/xyzF files

The additive principle is enforced by embedding structured comments. Standard XYZ parsers ignore comment lines, so they see only atom count, element symbols, and coordinates. Advanced parsers extract YAML metadata from comments.

3.2 Level 0: xyzZ — Raw Geometry

Standard XYZ format:

```
18
Water cluster (6 molecules)
O    0.0000  0.0000  0.0000
H    0.9572  0.0000  0.0000
H   -0.2400  0.9272  0.0000
O    3.0000  0.0000  0.0000
H    3.9572  0.0000  0.0000
H    2.7600  0.9272  0.0000
...
```

Line 1: atom count N . Line 2: comment (arbitrary text). Lines 3+: element symbol and Cartesian coordinates in Å.

This is the only format the engine *requires* as input. Any molecular visualization tool (VMD, PyMOL, Avogadro) can read it.

3.3 Level 1: xyzA — Annotated

Adds structured metadata via YAML in comment headers:

```
18
Water cluster | Generated by formation-engine v3.0.0
---
atoms:
  - id: a1
    element: O
    charge: -0.8
    mass: 15.999
  - id: a2
    element: H
    charge: 0.4
    mass: 1.008
  ...
bonds:
  - atoms: [a1, a2]
    order: 1
    type: covalent
  - atoms: [a1, a3]
    order: 1
    type: covalent
  ...
---
O      0.0000    0.0000    0.0000
H      0.9572    0.0000    0.0000
...
```

The YAML block (delimited by ---) contains:

- Atom IDs (deterministic: **a1**, **a2**, ...)
- Partial charges (from QEq or user specification)
- Bond table (inferred from distances)

Bonds are inferred via distance thresholding:

$$\text{bonded}(i, j) \iff |\mathbf{r}_i - \mathbf{r}_j| < f \cdot (r_{\text{cov},i}^{(k)} + r_{\text{cov},j}^{(k)}) \quad (22)$$

where $f = 1.2$ is a tolerance factor, $r_{\text{cov}}^{(k)}$ is the covalent radius for bond order $k \in \{1, 2, 3\}$, and k is selected to minimize the error $||r_{ij} - (r_i + r_j)|$.

This heuristic works well for typical organic and inorganic molecules but can fail for:

- Strained rings (distance criteria give false negatives)
- Metallic clusters (distance criteria give false positives)
- Radical species (unpaired electrons not captured)

The framework flags ambiguous cases (e.g., $|r_{ij} - (r_i + r_j)| > 0.3 \text{ Å}$) in the event log.

3.4 Level 2: xyzC — Constructed

Adds construction provenance: a complete record of operations applied to produce this structure.

A construction recipe is a sequence:

$$\text{recipe} = [\text{step}_1, \text{step}_2, \dots, \text{step}_n] \quad (23)$$

where each step is a named operation with parameters:

```
construction:
  source: unit_cell.xyz
  source_hash: a7f3c21b89d4e...
  steps:
    - op: supercell
      params: {a: 2, b: 2, c: 2}
      hash: 3c8f9d12a...
    - op: relax
      params: {method: FIRE, max_iter: 1000, ftol: 1e-6}
      hash: 9e4b2a1c7...
```

Each step carries a SHA-256 hash:

$$h_k = \text{SHA256}(\text{hash}_{k-1} \parallel \text{op}_k \parallel \text{params}_k) \quad (24)$$

The chain is append-only. If the source changes, `source_hash` changes, and all derived hashes are invalidated. This enables automatic rebuild detection: if the source file is modified, all derived files are marked stale.

Reserved Slots for Bulk Properties The xyzC format includes placeholders for properties that require ensemble statistics:

```
bulk_properties:
  density: null
  elastic_modulus: null
  rdf_reference: null

coarse_grained:
  bead_count: null
  bead_types: null
  pmf_reference: null
```

These are initialized to `null`. They must never contain default or placeholder values (e.g., `density: 1.0`) that could be mistaken for computed data. A `null` value is explicit: “this property has not been computed.”

When a property is computed (e.g., by running NPT dynamics and averaging the density), the field is updated:

```
bulk_properties:
  density: 0.997 # g/cm^3 (300 K, 1 atm)
  source: npt_run_12345
  uncertainty: 0.003
```

The `source` field records where the value came from. The `uncertainty` field records statistical error.

3.5 Level 3: xyzF — Force Annotated

Adds computed force vectors:

```
forces:
  model: LJ12-6 + Coulomb
  cutoff: 10.0
  timestamp: 2024-01-15T14:30:00Z
  hash: 7d9f3e2a1...

atoms:
  - id: a1
    net_force: [0.1234, -0.0567, 0.0892] # kcal/(mol)
    primary_interaction:
      partner: a2
      lj_component: [0.0800, -0.0400, 0.0600]
      coulomb_component: [0.0434, -0.0167, 0.0292]
      max_force: 0.1537
  - id: a2
    ...

statistics:
  max_force: 2.345
  mean_force: 0.123
  rms_force: 0.456
```

The primary interaction concept is key. For a 1000-atom system, displaying all $O(N^2)$ pairwise interactions is visual noise. Displaying only the dominant interaction per atom produces insight.

For atom i , the primary interaction is:

$$j^* = \arg \max_{j \neq i} |\mathbf{f}_{ij}| \quad (25)$$

where $\mathbf{f}_{ij}$ is the force on i from j . The xyzF format records:

- Partner atom j^*
- Lennard-Jones component $\mathbf{f}_{ij}^{\text{LJ}}$
- Coulomb component $\mathbf{f}_{ij}^{\text{Coulomb}}$

This answers: “What is the single strongest thing happening to this atom?”

For visualization, this is rendered as an arrow from atom i to atom j^* , color-coded by interaction type (red for repulsion, blue for attraction). This produces interpretable force diagrams even for large systems.

4 Hash Lineage and Provenance

Every derived state carries a cryptographic hash that serves as both identity and provenance key.

4.1 Hash Computation

The hash is computed as:

$$h = \text{SHA256}(\text{canonical}(\text{source}) \parallel \text{canonical}(\text{recipe})) \quad (26)$$

where $\text{canonical}(\cdot)$ produces a deterministic byte string:

- Floating-point values: 12 decimal places (e.g., 1.234567890123)
- Dictionary keys: sorted alphabetically
- Whitespace: normalized to single spaces
- Line endings: Unix-style (`\n`)

This ensures that the same physical operation produces the same hash regardless of:

- Platform (Windows, Linux, macOS)
- Locale (decimal separator)
- Timestamp (computation date irrelevant)

The hash is *stable under mathematically equivalent transformations*. For example:

$$\begin{aligned}\text{canonical}(\{a : 1.0, b : 2.0\}) &= \text{canonical}(\{b : 2.0, a : 1.0\}) \\ \text{canonical}(1.0) &= \text{canonical}(1.000000000000)\end{aligned}$$

But *unstable under physically distinct changes*:

$$\text{canonical}(1.0) \neq \text{canonical}(1.000000000001)$$

The 12-decimal-place precision is chosen to match typical floating-point error in energy minimization ($\sim 10^{-10}$ kcal/mol per atom).

4.2 Lineage Tracking

Hash lineage is append-only. Each construction step appends to the recipe and produces a new hash:

Step	Operation	Hash (first 8 chars)
0	Load unit cell	a7f3c21b
1	Supercell (2×2×2)	3c8f9d12
2	FIRE relax	9e4b2a1c
3	MD equilibration	1d5e7f3a

Given the final hash **1d5e7f3a...**, one can:

1. Look up the recipe in the metadata
2. Trace back to the source **a7f3c21b...**
3. Verify that re-running the recipe produces the same hash

This provides three guarantees:

Rebuild Detection If the source file changes, its hash changes. All derived files become stale. The build system detects this and triggers rebuilds.

Regression Identity The regression detector (Section 11.4) uses hashes to match configurations across parameter changes. If hash `9e4b2a1c` produces energy $E = -52.0$ kcal/mol with parameters Θ_1 and energy $E = -51.8$ kcal/mol with parameters Θ_2 , the difference $\Delta E = 0.2$ kcal/mol is a regression signal.

Provenance Auditing Any formation can be traced to its exact inputs. For a published dataset, the hash allows independent verification:

1. Download the source file
2. Run the recipe
3. Compare the output hash

If hashes match, the structure is reproduced. If not, either:

- The recipe is incomplete (missing parameters)
- The code has changed (different version)
- The source is wrong (different initial structure)

The hash makes non-reproducibility *detectable*.

5 Worked Example: State Evolution Through Operations

To make the abstract definitions concrete, consider a specific system: a 6-water-molecule cluster.

5.1 Initial State: Bare XYZ

Input file `water_cluster.xyz`:

```

18
Water cluster
O      0.0000   0.0000   0.0000
H      0.9572   0.0000   0.0000
H     -0.2400   0.9272   0.0000
O      3.0000   0.0000   0.0000
H      3.9572   0.0000   0.0000
H      2.7600   0.9272   0.0000
O      0.0000   3.0000   0.0000
H      0.9572   3.0000   0.0000
H     -0.2400   3.9272   0.0000
O      3.0000   3.0000   0.0000
H      3.9572   3.0000   0.0000
H      2.7600   3.9272   0.0000
O      1.5000   1.5000   2.5000
H      2.4572   1.5000   2.5000
H      1.2600   2.4272   2.5000
O      1.5000   1.5000  -2.5000
H      2.4572   1.5000  -2.5000
H      1.2600   2.4272  -2.5000

```

This is level 0 (xyzZ). The State at this point contains only:

- $N = 18$
- $\tau = [\text{O}, \text{H}, \text{H}, \text{O}, \text{H}, \text{H}, \dots]$
- $\mathbf{X} = [(0, 0, 0), (0.9572, 0, 0), \dots]$

Masses are inferred: $M_1 = 15.999$ (oxygen), $M_2 = 1.008$ (hydrogen), etc. Velocities, charges, bonds, forces, energies are undefined.

5.2 Operation 1: Annotate ($\text{xyzZ} \rightarrow \text{xyzA}$)

Command: `annotate-xyz water_cluster.xyz`

This operation:

1. Assigns atom IDs: **a1**, **a2**, ..., **a18**
2. Infers bonds from distances:
 - O–H bonds: $r_{ij} \approx 0.96 \text{ \AA} < 1.2 \times (0.66 + 0.31) = 1.16 \text{ \AA}$
 - H...O hydrogen bonds: $r_{ij} \approx 1.9 \text{ \AA}$ (not bonded, but noted in metadata)
3. Assigns partial charges via simple electronegativity:

$$q_{\text{O}} = -0.8e$$

$$q_{\text{H}} = +0.4e$$

Output: `water_cluster.xyzA`

The State now includes:

- $\mathbf{B} = \{(1, 2), (1, 3), (4, 5), (4, 6), \dots\}$ (12 O–H bonds)
- $\mathbf{Q} = [-0.8, 0.4, 0.4, -0.8, \dots]$

File size: 2.3 KB (was 0.5 KB for `xyzZ`; metadata adds 1.8 KB).

5.3 Operation 2: Evaluate Forces ($\text{xyzA} \rightarrow \text{xyzF}$)

Command: `compute-forces --model LJ+Coulomb --cutoff 10.0 water_cluster.xyzA`

This operation:

1. Zeros $\mathbf{F} \leftarrow \mathbf{0}$, $\mathcal{E} \leftarrow \mathbf{0}$
2. Evaluates Lennard-Jones forces on all pairs:

$$\mathbf{F}_i \leftarrow \mathbf{F}_i + \sum_{j \neq i} \mathbf{f}_{ij}^{\text{LJ}} \quad (27)$$

3. Evaluates Coulomb forces on all pairs:

$$\mathbf{F}_i \leftarrow \mathbf{F}_i + \sum_{j \neq i} \mathbf{f}_{ij}^{\text{Coulomb}} \quad (28)$$

4. Records primary interaction for each atom

5. Computes statistics: $F_{\max} = 2.34 \text{ kcal}/(\text{mol} \cdot \text{\AA})$, $F_{\text{rms}} = 0.45 \text{ kcal}/(\text{mol} \cdot \text{\AA})$

Output: `water_cluster.xyzF`

The State now includes:

- $\mathbf{F} = [(0.12, -0.06, 0.09), (-0.05, 0.02, -0.03), \dots]$
- $\mathcal{E} = (0, 0, 0, -12.3, -8.7, 0)$ (vdW and Coulomb only; no bonded terms because this is a non-bonded model)

File size: 5.8 KB (forces add 3.5 KB).

5.4 Operation 3: Minimize (`xyzF` $\rightarrow$ `xyzC`)

Command: `relax --method FIRE --max_iter 1000 water_cluster.xyzF`

This operation:

1. Initializes FIRE minimizer
2. Iteratively:
 - Evaluate $\mathbf{F}$ from current $\mathbf{X}$
 - Update velocities: $\mathbf{V} \leftarrow (1 - \alpha)\mathbf{V} + \alpha|\mathbf{V}|\hat{\mathbf{F}}$
 - Update positions: $\mathbf{X} \leftarrow \mathbf{X} + \Delta t \mathbf{V}$
 - Check convergence: $F_{\text{rms}} < 10^{-6}$?
3. Converges after 342 iterations
4. Records construction recipe:

```
construction:
  source: water_cluster.xyzA
  source_hash: d3e9f1a2...
  steps:
    - op: relax
      method: FIRE
      params: {max_iter: 1000, ftol: 1e-6, dt_init: 0.001}
      converged: true
      final_iter: 342
      final_energy: -21.045 # kcal/mol
      hash: 7c8a2d1f...
```

Output: `water_cluster.xyzC`

The State now includes:

- $\mathbf{X}$ (updated positions — molecules moved closer)
- Recipe with hash `7c8a2d1f...`

Final energy: $U_{\text{total}} = -21.0 \text{ kcal/mol}$ (stronger binding than initial configuration, which was $\sim -5 \text{ kcal/mol}$).

File size: 4.2 KB (construction metadata adds provenance but positions are similar).

5.5 Operation 4: MD Trajectory (xyzC $\rightarrow$ ensemble)

Command: `md-run --ensemble NVT --temp 300 --steps 10000 water_cluster.xyzC`

This operation:

1. Initializes velocities from Maxwell-Boltzmann at 300 K
2. Runs velocity Verlet integration with Berendsen thermostat
3. Saves snapshots every 100 steps (100 frames total)
4. Each snapshot is a State with updated $\mathbf{X}(t)$, $\mathbf{V}(t)$

Output: `water_cluster_traj.dcd` (binary trajectory file)

Analyzing the trajectory:

- Average temperature: $\langle T \rangle = 302.3 \pm 15.7$ K
- Average energy: $\langle U \rangle = -20.1 \pm 2.3$ kcal/mol (thermal fluctuations)
- Hydrogen bond count: 5.2 ± 0.8 (bonds form and break dynamically)

The trajectory ensemble is a collection of States $\{\mathcal{S}(t_1), \mathcal{S}(t_2), \dots, \mathcal{S}(t_{100})\}$, each with different $\mathbf{X}$ and $\mathbf{V}$ but the same N , $\mathbf{M}$, $\boldsymbol{\tau}$.

6 State Invariants and Validation

The State structure enforces several physical invariants that can be checked programmatically.

6.1 Invariant 1: Mass Conservation

$$M_{\text{total}} = \sum_{i=1}^N m_i = \text{const} \quad (29)$$

This must hold at every time step. If $M_{\text{total}}(t) \neq M_{\text{total}}(0)$, the integrator has a bug.

6.2 Invariant 2: Charge Neutrality (For Neutral Systems)

$$Q_{\text{total}} = \sum_{i=1}^N q_i = 0 \quad (\text{or specified total charge}) \quad (30)$$

For neutral molecules, $|Q_{\text{total}}| < 10^{-6} e$ (floating-point tolerance). Deviation indicates either:

- Incorrect charge assignment
- Numerical drift in a charge-equilibration scheme

6.3 Invariant 3: Momentum Conservation (NVE Only)

$$\mathbf{P}(t) = \sum_{i=1}^N m_i \mathbf{v}_i(t) = \mathbf{0} \quad (\text{if COM motion removed}) \quad (31)$$

In NVE dynamics with no external forces, $\mathbf{P}$ is conserved. Drift indicates violation of Newton's third law in the force evaluation.

Tolerance: $|\mathbf{P}(t) - \mathbf{P}(0)| < 10^{-10}$ (machine precision).

6.4 Invariant 4: Energy Ledger Self-Consistency

$$U_{\text{total}} = \sum_{k=1}^6 \mathcal{E}_k \quad (32)$$

The sum of ledger entries must equal the total potential energy computed independently. If not, either:

- A force model failed to report its contribution
- A force model reported to the wrong ledger entry

Tolerance: $|U_{\text{computed}} - U_{\text{ledger}}|/N < 10^{-10}$ kcal/mol per atom.

6.5 Invariant 5: Force-Energy Consistency (Finite Differences)

For any atom i and displacement $\delta \mathbf{x}$:

$$F_{i,\alpha} \approx -\frac{U(\mathbf{X} + \delta \mathbf{x} \mathbf{e}_\alpha) - U(\mathbf{X})}{\delta x} \quad (33)$$

where $\mathbf{e}_\alpha$ is the unit vector in direction $\alpha \in \{x, y, z\}$.

This is tested by finite differences with $\delta x = 10^{-5}$ Å. If the analytic force disagrees with the numerical gradient, the force derivation is wrong.

Tolerance: $|F_{\text{analytic}} - F_{\text{numeric}}| < 10^{-4}$ kcal/(mol·Å) (limited by numerical differentiation error).

7 State Transitions: The Physics Pipeline

Every physical operation is a State transformation:

$$\mathcal{S}(t + \Delta t) = \Phi_{\Delta t}[\mathcal{S}(t)] \quad (34)$$

where $\Phi_{\Delta t}$ is the time-evolution operator (integrator).

7.1 Integration as State Map

Velocity Verlet:

$$\Phi_{\Delta t}[\mathcal{S}] = \mathcal{S}' \quad \text{where:} \quad (35)$$

$$\mathbf{V}' \leftarrow \mathbf{V} + \frac{\Delta t}{2\mathbf{M}} \mathbf{F}(\mathbf{X}) \quad (36)$$

$$\mathbf{X}' \leftarrow \mathbf{X} + \Delta t \mathbf{V}' \quad (37)$$

$$\mathbf{F}' \leftarrow -\nabla_{\mathbf{X}'} U(\mathbf{X}', \mathbf{Q}, \mathbf{B}) \quad (38)$$

$$\mathbf{V}' \leftarrow \mathbf{V}' + \frac{\Delta t}{2\mathbf{M}} \mathbf{F}' \quad (39)$$

The integrator reads $(\mathbf{X}, \mathbf{V})$ from $\mathcal{S}$, computes $(\mathbf{X}', \mathbf{V}')$, and writes back to $\mathcal{S}'$. The identity fields $(N, \mathbf{M}, \boldsymbol{\tau})$ are copied unchanged.

7.2 Minimization as State Map

FIRE minimization:

$$\Phi_{\text{FIRE}}[\mathcal{S}] = \mathcal{S}^* \quad \text{where:} \quad (40)$$

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} U(\mathbf{X}, \mathbf{Q}, \mathbf{B}) \quad (41)$$

$$\mathbf{V}^* = \mathbf{0} \quad (42)$$

The minimizer finds the local minimum $\mathbf{X}^*$ and zeros the velocities (since we’re looking for a stationary point).

7.3 Composition of Operations

Operations compose naturally:

$$\mathcal{S}_{\text{final}} = \Phi_{\text{FIRE}} \circ \Phi_{\Delta t}^n \circ \Phi_{\text{init}}[\mathcal{S}_0] \quad (43)$$

This reads: “Initialize state, run n MD steps, minimize.”

Each operation is independently testable:

- Test Φ_{init} : Does it produce correct Maxwell-Boltzmann velocities?
- Test $\Phi_{\Delta t}$: Does it conserve energy in NVE?
- Test Φ_{FIRE} : Does it find local minima?

The composition inherits correctness from the components.

8 Memory Layout and Performance Considerations

The State structure is designed for cache efficiency.

8.1 Structure-of-Arrays (SoA) Layout

Positions are stored as three contiguous arrays:

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \quad (44)$$

$$= [x_1, y_1, z_1, x_2, y_2, z_2, \dots] \quad (45)$$

This is *array-of-structures* (AoS). Modern CPUs prefer *structure-of-arrays*:

$$\mathbf{X}_x = [x_1, x_2, \dots, x_N] \quad (46)$$

$$\mathbf{X}_y = [y_1, y_2, \dots, y_N] \quad (47)$$

$$\mathbf{X}_z = [z_1, z_2, \dots, z_N] \quad (48)$$

This enables vectorization: SIMD instructions (SSE, AVX) can process 4-8 coordinates in parallel.

The framework supports both layouts via compile-time flag. Default is AoS (for compatibility with standard tools), but SoA is available for performance-critical inner loops.

8.2 Memory Footprint

For $N = 1000$ atoms:

- Positions: $3 \times 1000 \times 8 = 24$ KB (double precision)
- Velocities: 24 KB
- Forces: 24 KB
- Masses: $1000 \times 8 = 8$ KB
- Charges: 8 KB
- Type IDs: $1000 \times 4 = 4$ KB (int)

Total: ~ 92 KB per State.

For trajectory storage, only positions (and optionally velocities) are saved. This reduces file size by $\sim 70\%$.

9 Conclusion: The State as Contract

The State structure is the contract between all components of the formation engine. It defines:

- **What must be conserved:** Identity fields never change
- **What must be computed:** Scratch fields regenerated on demand
- **What must be consistent:** Phase fields evolve under physical laws

This contract enables:

- **Modularity:** Force models and integrators are plug-and-play
- **Testability:** Each operation has clear inputs/outputs
- **Reproducibility:** States can be saved/restored exactly
- **Provenance:** Hash lineage tracks every operation

The result is a simulation framework where:

The State is the simulation. Everything else is just transformation.

This philosophical stance—that the data structure *is* the physics—permeates every design decision in the formation engine. It is why the State is defined formally at the beginning of the methodology document, rather than buried in implementation details. The State is not an engineering artifact. It is the formalization of phase space itself.

Transition to §3: With the State ontology established (the data structures that define a simulation), Section 3 documents *how* those States evolve: the physical interaction model that converts positions into forces and energies.